

## Core Banking Solution

---

### Scripting Syntax

---

10.2.25

# Contents

Scripting Syntax

Process Diagram

Processes

Technical Terms

About Scripting

Repositories and Classes

Return Status of the Script

Restrictions

Scripting Syntax

Comment

Operators

Arithmetic Operators

Comparison Operators

Logical Operators

Unary Operators

Expressions and Control Structures

Goto and Gosub

Calling and Starting another Script

Built-in Utility Functions

Executing User Routines

Executing User Executables

Adding to HTML Output

Executing Userhooks

Debugging Utilities

Exiting a Script

Encryption and Decryption of Scripts

Location of Script Files

Location of Sample Script Files

Executing a Script

Compiling a Script

Script Execution from FINRPT

Appendix

Summary

# Scripting Syntax Overview

This module discusses the scripting language which is provided as a customization tool, extensibility tool kit and also its syntax .

Scripting is a programming language that Finacle supports for various events. Scripting also allows formatting account numbers between Finacle and allied products like ATM, IBR and so on. The scripts for formatting account number across products and scripts for preload forms are pre-defined. All the scripts must exist at predefined locations. The users can only alter scripts as per their requirements.

As any other programming language, scripting has its own syntax.

Scripting can use certain variables, available from certain memory areas called as repositories. The user can within the script access the variables from the repositories.

Scripts also have built-in functions known as user hooks. Using user hooks the user performs the required tasks.

This module is organised into various sections. They include:

- [Process Diagram](#)

A diagrammatic representation of the process involved in scripting syntax.

- [Processes](#)

This section discusses the concepts critical for understanding the processes involved in scripting syntax.

- [Appendix](#)

A listing of the fields values available for scripts in the repository BANCS and class STDIN.

- [Summary](#)

A summation of the key concepts in scripting syntax.

## Scripting Syntax Overview

---

This module discusses the scripting language which is provided as a customization tool, extensibility tool kit and also its syntax .

Scripting is a programming language that Finacle supports for various events. Scripting also allows formatting account numbers between Finacle and allied products like ATM, IBR and so on. The scripts for formatting account number across products and scripts for preload forms are pre-defined. All the scripts must exist at predefined locations. The users can only alter scripts as per their requirements.

As any other programming language, scripting has its own syntax.

Scripting can use certain variables, available from certain memory areas called as repositories. The user can within the script access the variables from the repositories.

Scripts also have built-in functions known as user hooks. Using user hooks the user performs the required tasks.

This module is organised into various sections. They include:

### [Process Diagram](#)

A diagrammatic representation of the process involved in scripting syntax.

### [Processes](#)

This section discusses the concepts critical for understanding the

processes involved in scripting syntax.

## [Appendix](#)

A listing of the fields values available for scripts in the repository BANCS and class STDIN.

## [Summary](#)

A summation of the key concepts in scripting syntax.

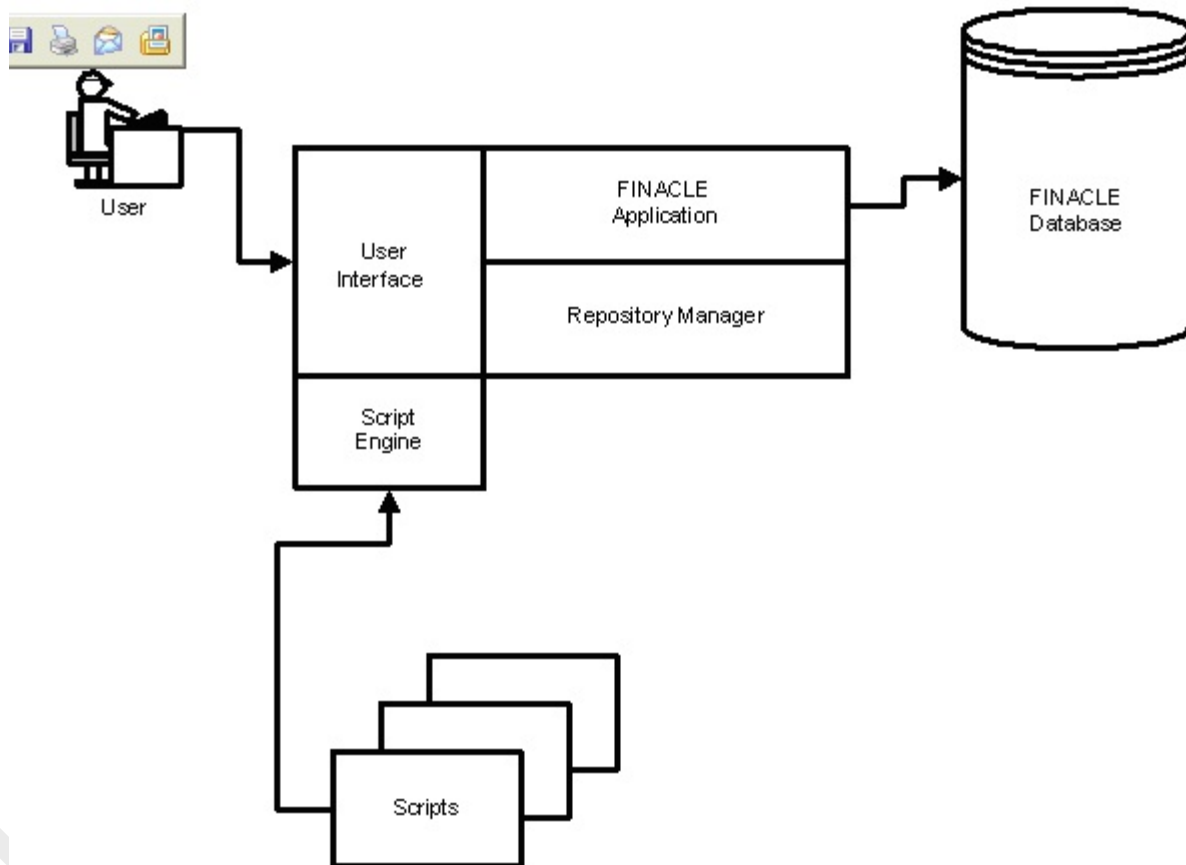
© EdgeVerve Systems Limited

External

Confidential

## Process Diagram

The process diagram for scripting syntax is as follows:



## Processes

---

This section discusses the concepts critical for understanding the processes involved in scripting syntax, including:

- [Technical Terms](#)
- [About Scripting](#)
- [Repositories and Classes](#)
- [Return Status of the Script](#)
- [Restrictions](#)
- [Scripting Syntax](#)

© EdgeVerve Systems Limited

External

Confidential



## Technical Terms

---

The technical terms relevant in the context of scripting syntax are as follows:

### Repository Manager

This module provides the capability of storing variable values and identifying these variables by a name. This module is used by the script engine to store values and transfer them between menu options. The structure of a repository field reference is as follows:

[Repository Name].[Class Name].[Field Name]

### Queue Table

This is the area where uncompleted workflow is stored temporarily, until retrieved for further processing. On completion of the workflow, the stored data is deleted and an audit record is created optionally.

### TBA\_SCRIPTS

TBA\_SCRIPTS is an environment variable set in the commonenv file. This is the directory where all the scripts should exist. The application will look for this directory **\$TBA\_PROD\_ROOT/cust/01/INFENG/scripts**. If the script is not available in this directory, it will pick the script from the directory **\$TBA\_PROD\_ROOT/prodbase/INFENG/scripts**. If the script is also not available in this directory also, the default logic (if available) will be applied as specified in each case.

## Repository

A repository is a memory structure to store data of the user. Scripts access data from repository only. Repository Structure is user based.

## Userhook

A pre-defined function defined in Finacle to perform necessary tasks. Using Userhooks the user can achieve certain required operations in a script.

© EdgeVerve Systems Limited

External

Confidential

## About Scripting

---

Scripting is a programming language supported by Finacle for manipulating a set of defined input data in a script and give a defined output data for further processing. This provides more flexibility to the end user in manipulating the data entered in the front end to suit the needs before the back end processes it. Scripting is also used to solve the usability issues in the user interface and for interfacing with other delivery channels.

---

### **For Example:**

Account numbers can be defined according to the Bank's requirement. The format defined can be made common for all the accounts in the Service Outlet. In this case, the user may not want to enter the common portion of characters every time when account number field is to be entered.

Using Finacle Scripting, it is possible for the bank to get the common portion appended to the account number irrespective of the menu option being executed. It can be the Bank code, Branch code, Currency code, currency alias, Data center, etc., which are common to all accounts.

The account number available in the database might be in the format 01-02-03-04-0001:

Where 01 - corresponds to Bank code, 02 - corresponds to Branch code, 03 - corresponds to the currency alias, 04 - corresponds to the scheme code and 0001- is the actual account number.

If this account number has to match, the value entered in the front end by the user, then he has to type it in the same order

which is very cumbersome.

To make this easier, the user will type the account number without the '-' symbol and using scripts the account number can be changed to a format acceptable by the backend.

---

### **For Example:**

The branch may be connected to an ATM that supports only 10 character account number and the actual account number size in Finacle may be more. In this case, the Finacle account number format is to be converted to ATM account number format using some logic. This can be done in scripting. The bank will decide on the logic to derive the unique account number for ATM from Finacle format and codes the same in the script.

The account number in Finacle may be 01-02-03-04-0001.

Where 01 - corresponds to Bank code 02 - corresponds to Branch code 03 - corresponds to the currency alias 04 - corresponds to the scheme code 0001 is the actual account number.

To convert to 10 character ATM account number format, the bank may decide to leave out the bank code portion as the ATM is attached to the branches of the same BANK and leave out the formatting of account number with '-'.

The ATM account number that is derived can be 0203040001 that must be unique in the data center.

---

## Repositories and Classes

---

Scripting implements a way of passing data between the Finacle programs, user hooks, user routines, user executables and the script using repositories.

A Repository is a named entity, which holds a set of named Classes.

A Class is a set of name, value pairs. The name, value pair is called a fieldname and fieldvalue. A Class can hold only one type of fieldvalue, which is specified during Class creation.

Integer, Float, Double, Character and String Classes are currently supported. A fieldvalue is referenced using the syntax: Repository-name.Class-name.Field-name

Some standard repositories and classes are managed by Finacle and are available for use.

See [Appendix](#) for a list of repositories, classes and variables available.

The bank can also create its own repositories, classes and fields while implementing the scripting logic. However, it is their responsibility to cleanup these repositories after use. Else, they may occupy memory unnecessarily.

The INPUT CLASS and OUTPUT CLASS contain fields specific to the events.

You can do the following:

- [Manipulate Repository Variables](#)

## Return Status of the Script

---

One of the standard output fields from the script is `successOrFailure`. In case a FAILURE is encountered in the script, the OUTPUT CLASS variable `successOrFailure` is set to 'F' as shown:

```
BANCS.OUTPUT.successOrFailure = " F"
```

The valid values are "S" for success and " F" for failure.

The default return status from the script is "S".

© EdgeVerve Systems Limited

External

Confidential

## Restrictions

---

There is a size limit of 128 KB for any Finacle script file. If a script is greater than this size then a fatal error will be thrown.

---

### **For Example:**

A script file LoanDmdSatisfy.scr of size more than the maximum limit was created and this was the error thrown during execution.

**File [./ LoanDmdSatisfy.scr] greater than 131072**

---

© EdgeVerve Systems Limited

Confidential

External

## Scripting Syntax

---

Every script must begin with a '<--start' and end with a 'end-->' tag.

Script has 26 built-in scratch pad variables. These are sv\_a, sv\_b, ... sv\_z.

All these variables are globally accessible across scripts. The user need not define the scratch pad variables to be of a particular type. The assignment to a scratchpad variable will decide the type of the variable based on the type of the right-hand side. Scripting allows the dynamic type change.

### Local Variables:

Script has local variables whose scope is limited within the script and not automatically reflected to the called scripts.

The name for local variables is prefixed by "lv\_" and is ended by any single alphabet. Hence, maximum 26 local variables can be used in a particular script. For example lv\_a through lv\_z.

The scratch pad variables declared in the calling scripts are visible and accessed in the called script whereas local variables cannot be accessed in the called script.

This type is maintained until a new assignment is made. So Scripting allows the dynamic type change.

---

### For Example:

```
sv_a = 12.45
```



.....

....

sv\_a = "Hello"

makes the type of 'sv\_a' to be set to DOUBLE type at the first line. Until the last line sv\_a carries its type as DOUBLE. In the last statement its type is set to STRING.

---

**Note:**

- 1.Ensure that the first line of the script is the start tag. Though this is not essential, it helps in debugging scripts when errors are encountered as script counts line numbers from the start tag. It is recommended to have all comments after the start tag only.*
- 2.Start tag is mandatory but end tag is not. Its always recommended to have end tag as well.*

This section also discusses:

- [Comment](#)
- [Operators](#)
- [Expressions and Control Structures](#)
- [Goto and Gosub](#)
- [Calling and Starting another Script](#)
- [Built-in Utility Functions](#)
- [Executing User Routines](#)
- [Executing User Executables](#)
- [Adding to HTML Output](#)
- [Executing Userhooks](#)
- [Debugging Utilities](#)

- [Exiting a Script](#)
- [Encryption and Decryption of Scripts](#)
- [Location of Script Files](#)
- [Location of Sample Script Files](#)
- [Executing a Script](#)
- [Compiling a Script](#)
- [Script Execution from FINRPT](#)

© EdgeVerve Systems Limited

External

Confidential

## Comment

---

Scripting allows comments in scripts.

All statements with '#' as the first non-white space character are treated as comments. This is the only way to make a line as comment.

Even if a sequence of lines are to be treated as comments, each line must be preceded with a '#'.

© EdgeVerve Systems Limited

External

Confidential

## Operators

---

Scripting language supports the following operators:

- [Arithmetic Operators](#)
- [Comparison Operators](#)
- [Logical Operators](#)
- [Unary Operators](#)

© EdgeVerve Systems Limited

Confidential

External

## Arithmetic Operators

---

'+', '-', '\*', '/'.

Operator '+' works both for STRING/CHAR type also.

'-', '\*', '/' work only for numeric type variables.

```
<--start  
sv_a = "FIRST"  
sv_b = sv_a + " AND "  
#sv_b is "FIRST AND "  
sv_c = "SECOND"  
sv_a = sv_b + sv_c  
#now sv_a is "FIRST AND SECOND"  
sv_a = "FIRST" + " AND " + "SECOND"  
# sv_a is "FIRST AND SECOND"  
sv_a = 10  
sv_b = 5  
sv_c = sv_a + sv_b  
# sv_c is 15  
sv_d = sv_a / sv_b  
# sv_d is 2
```

```
sv_e = sv_a - sv_b  
# sv_e is 5  
sv_f = sv_a * sv_b  
# sv_f is 50  
end-->
```

© EdgeVerve Systems Limited

External

Confidential

## Comparison Operators

---

'==', '<=', '>=', '<', '>', '!='

The outcome all these operations are TRUE (1) or FALSE (0).

In all these operations, if both arguments are STRINGS the string comparisons are made else, the STRINGS are converted to INTEGERS and the comparisons are done. If CHAR types are available they are accessed as INTEGER types and comparisons are made.

```
<--start
sv_a = 10
sv_b = 12
if (sv_a == sv_b) then
    print ( "sv_a is equal to sv_b")
else
    print ( "sv_a is not equal to sv_b")
endif
if (sv_a != sv_b) then
    print ( "sv_a is not equal to sv_b")
else
    print ( "sv_a is equal to sv_b")
```

```
endif
if (sv_a < sv_b) then
    print ( "sv_a is less than sv_b")
else
    print ( "sv_a is not less than sv_b")
endif
end-->
```

© EdgeVerve Systems Limited

External

Confidential



## Logical Operators

---

- 'AND', 'OR'

The outcome of all these operations are TRUE (1) or FALSE (0).

In all these operations, if both arguments are STRINGS, the string logical operations are made. If CHAR types are available they are accessed as INTEGER types and logical operations are made.

### Warning:

In Scripts whenever there is a Logical AND/OR separating two conditions, these conditions must be separated with parenthesis from the AND/OR keyword. Only then the proper evaluation of the conditions occurs.

---

### For Example:

To do a processing only when the values in variable sv\_a and sv\_b are not null the condition should be written as

```
if ( (sv_a != "") AND (sv_b != "") ) then
```

```
-----do processing -----
```

```
endif
```

it should not be written as

```
if(sv_a != "" AND sv_b != "")
```

This does not evaluate properly.

```
<--start
```

```
sv_a = "Hello"
sv_b = "Hi"
if (sv_a != "" AND sv_b != "") then
    print ( "Does not evaluate properly")
endif
if ( (sv_a != "") AND (sv_b != "") ) then
    print ( "Both are not null")
endif
sv_a = 10
sv_b = 0
if (sv_a AND sv_b != "") then
    print ("Doesn't work")
endif
end-->
```

---

**Note:**

Ensure comparisons are made between data of the same data type only (strings, integer and so on).

© EdgeVerve Systems Limited

Confidential

External

## Unary Operators

---

The same '-' operator can be used as binary as well as unary operator.

UNARY operator can only be used with numeric types.

```
<--start  
sv_a = 10  
sv_b = -sv_a  
# Now sv_b is -10  
end-->
```

© EdgeVerve Systems Limited

Confidential

External

## Expressions and Control Structures

---

In scripting, expressions are valid combinations of variables, literals and operators.

Scripting provides the following control structures:

### Checking a condition:

\* IF (condition) THEN

statements

ENDIF

\* IF (condition) THEN

statements

ELSE

statements

ENDIF

These two statements can be used depending on the necessity. Nested IF conditions are allowed and the execution of these statements depends on the outer loop condition.

```
<--start
```

```
sv_a = 10
```

```
sv_b = 12
```

```
if (sv_a > sv_b) then
```

```
        print ( "sv_a is greater than sv_b")
else
        print ( "sv_a is less than or equal to sv_b")
endif
end-->
```

Each statement should be in a new line.

### **Looping :**

\* WHILE (condition)

statements

DO

The above statement executes the statements until the condition in the WHILE becomes FALSE (0).

Each statement must be in a new line.

```
<--start
sv_s = 1
print (sv_s)
while (sv_s <= 1000)
    sv_s = ((sv_s + (10*(2+3))) + 23)
    print (sv_s)
do
end-->
```

## Goto and Gosub

---

Transfer of control from one part of the script to another in the same script file is facilitated by two constructs provided in the scripting. The two constructs are GOTO and GOSUB.

The syntax is as follows:

- GOTO LabelName
- GOSUB LabelName

The LABEL referred by LabelName in GOTO statement can only be forward referenced, GOTO cannot reference a label that exists before this statement in the script.

In case of GOTO the execution of the script starts from the statement where the LABEL is declared.

The LABEL referred by LabelName in GOSUB statement can be anywhere in the script. The label can be either before or after this GOSUB statement

In case of GOSUB the execution of the script starts from the statement, where the LABEL is declared and returns back to the next statement after the GOSUB as soon as it finds RETURN statement in the script.

### **Note:**

GOSUB cannot be used inside the control structure to point to a SUBROUTINE that is outside the innermost control structure.

The following script gives an error:

```
<--start  
sv_a = 1  
# if condition starts here  
If(sv_a == 1) THEN  
    GOSUB subRoutine1  
# if condition ends here  
ENDIF  
EXITSCRIPT  
# sub routine is outside the if-endif condition  
subRoutine1:  
    print (sv_a)  
    RETURN  
end-->
```

Following SCRIPT will work fine.

```
<--start  
sv_a = 1  
# if condition starts here  
if(sv_a == 1) THEN  
    GOSUB subRoutine1  
# we added a GOTO statement to jump  
# beyond the subroutine after the sub  
# routine exits
```

```
GOTO jmp1
# sub routine is inside the if-endif condition
subRoutine1:
    print (sv_a)
    RETURN
jmp1:
# if condition ends here
endif
EXITSCRIPT
end-->
```

© EdgeVerve Systems Limited

External

Confidential



## Calling and Starting another Script

---

Transfer of control from one script to another is facilitated by two constructs provided in the Scripting. The two constructs are CALL and START.

The syntax is:

CALL ( ScriptName )

START ( ScriptName )

where ScriptName is either a string, like, “ script1.scr ”, or Repository variable or Scratchpad variables of STRING type.

In both the above statements, if the script file does not exist in the PATH, an error is reported.

CALL ( Path, ScriptName )

START ( Path, ScriptName )

where Path is either a string or a Repository variable or scratchpad variable of STRING type,

ScriptName is either a string or a REP variable or scratchpad variable of STRING type

Path and ScriptName are appended to get the full path.

If the script file exists and is readable, the execution of the file starts from the beginning.

In case of CALL the execution of the script returns back to the same

point in the old script as soon as it finds EXITSCRIPT statement in the new script.

All the scratchpad variables have a global context. That is, all the scratchpad variables are carried over from the caller script to the called script and vice-versa.

In case of `START` the execution of the script moves from the first script to the second and stops after the execution of the second script.

**EXAMPLE for START:**

```
<--start
trace on
sv_a = 1
print (sv_a)
while (sv_a < 100)
    sv_a = sv_a + 2
    print (sv_a)
do
start ("called.scr")
if (sv_a > 100) then
exitscript
endif
trace off
end-->
called.scr
<--start
```

```
trace on
print ("Hello World. Happy new Year")
exitscript
trace off
end-->
```

© EdgeVerve Systems Limited

External

Confidential

## Built-in Utility Functions

Script can use built-in functions at appropriate places in expressions. The evaluation of these functions results as follows:

| Built-in Functions                 | Results                                                                                   |
|------------------------------------|-------------------------------------------------------------------------------------------|
| MID\$ (Var, StartPosition, Length) | MID\$ (Var, StartPosition, Length)                                                        |
| LEFT\$ (Var, Length)               | Returns the leftmost Length number of bytes from Var.                                     |
| RIGHT\$ (Var, Length)              | Return the rightmost Length number of bytes from Var.                                     |
| CINT (Var)                         | Convert Var to an integer.                                                                |
| CDOUBLE (Var)                      | Convert Var to a double.                                                                  |
| TOLOWER (Var)                      | Convert all character(s) of STRING/CHAR to lowercase.                                     |
| TOUPPER (Var)                      | Convert all character(s) of STRING/CHAR to uppercase.                                     |
| FORMAT\$ (Var, FormatString)       | Formats the contents of Var according to the FormatString specified in the C print style. |
| SET\$ (Var1, From, Length, Var2)   | Sets the contents of Var1 from From position till Length bytes to the content of Var2.    |
| ENCRYPT (Var)                      | Encode Var into MIME format " x-www-form-urlencoded".                                     |
| STRLEN (Var)                       | Returns the length of Var.                                                                |
| CHARAT (Var, Position)             | Returns the character at Position in Var.                                                 |
| LTRIM (Var1 [, Var2])              | Left trim Var1. Var2 is the character to be trimmed. Default value of Var2 is ' '.        |

|                                |                                                                                                                                                                               |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RTRIM (Var1 [, Var2])          | Right trim Var1. Var2 is the character to be trimmed. Default value of Var2 is ' '.                                                                                           |
| TRIM (Var1 [, Var2])           | Trim Var1. Var2 is the character to be trimmed. Default value of Var2 is ' '.                                                                                                 |
| LPAD (Var1, Var2 [, Var3])     | Left pad Var1 with Var3 upto the length Var2. Var3 is the character to be used for padding. Default value of Var3 is ' '.                                                     |
| RPAD (Var1, Var2 [, Var3])     | Right pad Var1 with Var3 to make the length Var2. Var3 is the character to be used for padding. Default value of Var3 is ' '.                                                 |
| REPEXISTS (Var1)               | Checks if Repository Var1 exists.                                                                                                                                             |
| CLASSEXISTS (Var1, Var2)       | Checks if Class Var2 exists in Rep Var1.                                                                                                                                      |
| FIELDEXISTS(REP.CLA.FLD)       | Checks if the Field 'FLD' exists in the Class 'CLA' which exists in the Repository 'REP'.                                                                                     |
| GETPOSITION (Var1, Var2)       | Returns the first position of Var2 in Var1. Var1 is String and Var2 is String/Char. Case Sensitive.                                                                           |
| GETIPOSITION (Var1, Var2)      | Returns the first position of Var2 in Var1. Var1 is String and Var2 is String/Char. Case Insensitive                                                                          |
| STRICMP (Var1, Var2)           | Does String Comparison of two Strings/ Characters without regard to case.                                                                                                     |
| GETSTRING (Var1)               | Converts a Char type to String Type.                                                                                                                                          |
| CREATEREP (Var1)               | Create a Temporary Repository Var1.                                                                                                                                           |
| CREATECLASS (Var1, Var2, Var3) | Create a Temporary Class Var2 in Repository Var1 of the type Var3. Var3 has the following value<br>1 for INTEGER<br>2 for DOUBLE<br>3 for FLOAT<br>4 for CHAR<br>5 for STRING |
| DELETEREP (Var1)               | Delete the Temporary Repository Var1                                                                                                                                          |
| DELETECLASS (Var1, Var2)       | Delete the Temporary Class Var2 in Repository Var1                                                                                                                            |

|                        |                                                          |
|------------------------|----------------------------------------------------------|
| PUTREPFIELD(Var1,Var2) | Puts the contents of Var1 into a field specified by Var2 |
| GETREPFIELD(Var1)      | Returns the value of the field specified by Var1         |

*For examples of utility function see [Example](#).*

© EdgeVerve Systems Limited

External

Confidential

## Executing User Routines

---

The scripting language provides the user with a facility to call user routines, user hooks and user executables from within the script.

User routines are written and compiled into a DLL/so. Default DLL/so is SERTN.DLL.

The following is the syntax for specifying the name of the DLL/so within a script:

`LIBNAME lib`

where `lib` is the name of the DLL/so which must be in the `PATH`. In UNIX, the environment variable `LD_LIBRARY_PATH` has to be modified to have the path in which the “so” is present.

The following is the syntax for calling a user routine within a script:

`sv_a = URTN_FunctionName ( STRING )`

where `sv_a` will have the value returned from the function `FunctionName`,

`FunctionName` is the function that exists in the `lib` and

`STRING` is either a string, like, “Data”, or a REP variable or scratchpad variable of `STRING` type.

The interface for the user routines is

```
int UserRtn ( IW_WEB_EXE_ENV_PTR WExeEnvPtr, void *  
SEHandle, char * InBuff, char ** OutBuff, int * iLen, int * iRet)
```

where InBuff is the input parameter to the function passed above (STRING),

OutBuff is the buffer to be appended to the output buffer,

iLen is the length of the buffer returned,

iRet is the value to be returned by the function.

**Note :**

*Do not call SEStartScriptWExe or SEStartScriptExe from user routines. If another script is called from the user routines, then the return value of the user routines is not returned in the LVALUE. The LVALUE has the same value as before calling the function.*

© EdgeVerve Systems Limited

External

Confidential



## Executing User Executables

---

User executables are used to execute other executables, read the data dumped by them and populate the appropriate Rep, Class and Fields.

The following is the syntax for calling a User Executable within a script:

```
sv_a = UREX ( ExecutableName, ReturnDataFile, Arg )
```

where sv\_a will have the exit value of the Executable, ExecutableName,

ExecutableName is the Executable's name and is either a string, like "Data", or a REP variable or scratchpad variable of STRING type

ReturnDataFile is the file in which the ExecutableName dumps all the return data. It is either a string, like "Data", or a REP variable or scratch pad variable of STRING type.

Arg is the argument that is passed to the ExecutableName. It is either a string, like, "Data", or a REP variable or scratchpad variable of STRING type.

The ExecutableName is called with two parameters. First argument is ReturnDataFile while the second is Arg.

The exit value of the ExecutableName is returned in the left hand variable, in this case sv\_a. The following exit values are reserved by the Script Engine :

- 250 ( SE\_EXEC\_FAILED) - Exec failed
- 251 ( SE\_EXEC\_GAVE\_CORE) - Executed process gave a core
- 252 ( SE\_EXEC\_STOPPED) - Executed process stopped

The ExecutableName dumps all the return data in REP.CLASS.FIELD=VALUE format separated by ('\ n' or '\ t' or ' ') in the file ReturnDataFile. The Script Engine parses the file ReturnDataFile and populates the appropriate Repositories with the data. The type of Repository is found according to following set of rules :

If VALUE starts with ' ' , then it is of type CHAR,

If VALUE starts with ' " ' , then it is of type STRING and is terminated by closing ' " ' ,

If VALUE contains ' . ' , then it is of type DOUBLE/FLOAT. Script Engine gets the type of the CLASS and puts the value appropriately,

Else the VALUE is of type INTEGER.

If the Repository or Class does not exist, then an error is set. The user is supposed to create the ReturnDataFile in case it doesn' t exist. After parsing, ReturnDataFile is removed by the Script Engine.

© EdgeVerve Systems Limited

External

Confidential

## Adding to HTML Output

---

To directly add to the HTML output buffer from the ScriptEngine, use the command, `AddToHTML (STRING)`, where `STRING` is either a string, like “Data”, or a REP variable or scratchpad variable of ANY type.

© EdgeVerve Systems Limited

External

Confidential

## Executing Userhooks

---

Scripting provides for certain functions, which can be called within a script. These are known as user hooks.

The following is the syntax for calling a user routine within a script:

```
sv_a = URHK_FunctionName (STRING)
```

where sv\_a will have the value 0(TRUE) if the user hook is successful or else it will be 1(FALSE) for failure.

FunctionName is the user hook and STRING is either a string, like "Data", or a repository variable or scratchpad variable of STRING type.

There are default scripting user hook functions available for use in the script. These hook functions provide the functionality and the input and output parameters are defined.

© EdgeVerve Systems Limited

External

Confidential

## Debugging Utilities

---

The application programmer can debug the application by setting the TRACE option ON. To set the trace off, TRACE OFF can be used. By default, trace is OFF. Setting TRACE ON, makes the script engine log all the information about each statement it has executed in a log file by name ScriptFile\_PID.trc.

The trace file generation also depends on the existence of the .usertraceon file in the user home directory. If this file is present then irrespective of whether trace is on or off the trace file is generated. If the file .usertraceon is not present and TRACE is OFF, then no trace file is generated.

The trace file is formed in the directory depending on:

- The trace path is read from environment variable SE\_TRACE\_PATH. If the environment variable is not set or there is no write permission for the path specified then
- ..\log is taken as the log path. If there is no write permission for the specified path then, current directory is taken as the log path. If there is no write permission for the specified path then an error is set

Within the script, to log information about a part of the script we can use the combination of these two commands.

---

**For Example:**

.....

.....

TRACE ON

.....

.....

TRACE OFF

.....

---

Scripting also provides another important command to print a particular Built-in variable value or REP/LL field value.

The syntax of PRINT command is:

PRINT ( SV\_S )

(or)

PRINT ( "My String" )

(or)

PRINT ( GLOBAL.SECLASS.OUTPUTDATA )

Similar to .usertraceon file there exists a file .userprinton. The three possible cases for this are as follows:

- The output of PRINT tokens is diverted to a file named "<Filename>\_<PID>.stdout" – if .userprinton file is existing in the user's home directory
- All output of PRINT tokens diverted only to trace file - if TRACE is ON (or .usertraceon file exists)
- All output of PRINT tokens diverted to console (stdout) - If TRACE is OFF

**NOTE:**

*A feature has been provided to disable all script tracing. This will override the tracing option enabled at script level. To disable scripting*

*an environment variable called **GLOBAL\_TRACE** has to be set to **OFF**. (Command : export GLOBAL\_TRACE=OFF). This has to be set before starting finlistval services. (This can be put in the start file of finlistval bin ).If this environment variable is set, the userhook B2k\_PrintRepos will also not generate the trace.*

– Setup changes:

GLOBAL\_TRACE has to be exported in the startup file for finlistval with value as **OFF**.

© EdgeVerve Systems Limited

External

Confidential

## Exiting a Script

---

Scripting provides a safe way of exiting the execution of the script. Whenever the Script Engine finds EXIT statement in the execution path stops the execution and returns to the Calling routine.

Even when the EXIT statement is encountered in a new script that has been CALLED from another script, Script Engine stops the whole execution and returns to the upper layer.

To return from CALLED script to the calling script, use EXITSCRIPT.

See [Executing a Script](#) for more details.

© EdgeVerve Systems Limited

External

Confidential



## Encryption and Decryption of Scripts

---

The script files are ASCII text files and can be looked at and even modified at the Finacle installations. These scripts must be protected against modifications in the installations. Hence, encryption is provided using which the bank can write scripts and encrypt them before putting them into live installations. Scripting automatically detects if a file is encrypted and decrypts it if required.

### Encryption/decryption utility [bauu9375]

**Usage :** bauu9375 <Input File Name> <Output File Name> <mode E/D>

---

#### For Example:

```
$bauu9375 bg_issue1.scr bg_issue.scr E
```

where <Input File Name> is the input file to be encrypted/decrypted,

<Output File Name> is the output file,

<mode E/D> is the mode, E (encryption) and D (decryption).

---

The utility warns if the user tries to encrypt a file that has already been encrypted or tries to decrypt a file that is already decrypted.

## Location of Script Files

---

Scripting locates the script file to execute, using the same logic as Finacle for its executables. The order of searching script files is as follows:

- \$TBA\_PROD\_ROOT/cust/<entity id>/ <lang code> /scripts
- \$TBA\_PROD\_ROOT/cust/<entity id>/INFENG/scripts
- \$TBA\_PROD\_ROOT/cust/<lang code>/scripts
- \$TBA\_PROD\_ROOT/cust/INFENG/scripts
- \$TBA\_PROD\_ROOT/prodbase/<lang code>/scripts
- \$TBA\_PROD\_ROOT/prodbase/INFENG/scripts

Where <entity id> is the bank ID and <lang code> is the language code.

© EdgeVerve Systems Limited

External

Confidential

## Location of Sample Script Files

---

The sample scripts can be located in **\$TBA\_PROD\_ROOT/sample/samscripts** directory. These scripts are for sample purpose only hence they have different file extension - sscr (stands for sample scripts) and not scr which is for usual scripts. Developers can use them as templates to start with. For this, the sample script must be copied to **\$TBA\_PROD\_ROOT/cust/01/INFENG/scripts** directory followed by the relevant modifications. At the end, the extension must be changed from sscr to scr.

See [Executing a Script](#) for more details.

© EdgeVerve Systems Limited

External

Confidential

## Executing a Script

The Core Banking Solution provides the HSCRIPT menu option to test Finacle scripts. It brings up a screen, which accepts a script name and executes it. Repository variables are used to store the input and output values of the script. The default input values are provided in the script and are stored in a class called 'INPUT' under BANCS repository. The output obtained is stored in the class 'OUTPUT' and can be printed in the trace file.

The pre script can be used to initialize, set values for variables and the post script to do processing based on the values derived from script.

You can invoke the HSCRIPT menu option.

### What you see:



Click **Submit**

On selecting **YES** for create repositories report field, a report of all the default repositories are created

### Note:

*The report created will not list the repositories created by the user.*

## Compiling a Script

---

### Compiler Tool

**Exename:** scrtok

**Usage:** scrtok <script filename>

**Purpose:**

This compiler tool is a utility (executable) which compiles or tokenizes the given script file and stores it as a tokenized binary file. The name of the compiled file generated must be <script name>.tok

---

**For Example:**

If the name of the script is First.scr, then the name of the compiled file must be First.scr.tok

---

Once a script is compiled, the compiled file can be deployed in lieu of the script. Deploying a compiled script file ensures faster execution of the script compared to plain text script file.

**File renaming:**

Before using the compiled file instead of the script, the **.tok** file has to be renamed back to **.scr** file for the application to be able to locate the script file.

**Conditions where compiling will fail:**

The conditions under which script compiling fails are:

■ Either 'trace on' or 'trace off' is used in the script.

■ The script has specific syntactical errors leading to unknown tokens.

---

**Example:**

If in the script the rep is created as the following ( rep.scr)

```
<--start
```

```
sv_d = REPEXISTS("SARAV")
```

```
if (sv_d == 0) then
```

```
CREATEREP ('SARAV")
```

```
Endif
```

```
end-->
```

This is a syntax error.

If scrtok rep.scr

The following error is thrown

Error Description: Error in tokenizing the Script

Filename [tmp\_SETokenizer.c]

Line number [71]

---

## Script Execution from FINRPT

---

“**Script execution from FINRPT**” is a feature that allows executing a Finacle Script from FINRPT templates for fetching additional information from the database or application memory. This feature is useful when there is a requirement of printing some data in the report which is not available in the data file or repository files.

### Utility for script execution:

A method “executeScript” is available in the Finacle Core specific utility class FinCoreUtil. This method can be used to call a Finacle script within the report template.

#### **Note:**

*Detailed information about this method is available in the form of JAVADOC through menu option “Plugins->Help->FINRPT In-built functions Javadocs” in the Finacle Report Designer.*

### Method signature:

```
public static boolean  
executeScript(java.lang.String scriptName,  
              java.lang.String  
inputData,  
              java.lang.String  
outputData,  
              com.infy.finacle  
finScriptlet,
```

FinAppUtil

finAppUtilInst)

throws

FinRptCallbackFailedException

This method executes the given Finacle Script and populates output of script in to the given report variables. The call to this method can be included as an expression of a report element like TextField or a report variable.

### Usage:

```
FinCoreUtil.executeScript(<scriptName>,  
<inputData>,  
<outputData>,  
$P{REPORT_SCRIPTLET},  
$P{FIN_APP_UTIL})
```

### Parameters:

| Parameter Name               | Description                                                                                                                                                                                                                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>scriptName</b>            | Name of script to be executed                                                                                                                                                                                                                                                                                                                              |
| <b>inputData*</b>            | <p>Input data for script execution. The format of the this field could be any of the below formats:</p> <p>fld1=val1 fld2=val2 ...</p> <p>fld1=val1 var2 ...</p> <p>var1 var2 ...</p> <p><b>Note:</b></p> <p>Here var1,var2, etc. are the names of variables defined in report and value of the same would be updated with output of script execution.</p> |
| <b>\$P{REPORT_SCRIPTLET}</b> | The in-built report scriptlet parameter. This should be passed as it is. It is used for updating values of output variables.                                                                                                                                                                                                                               |
| <b>\$P{FIN_APP_UTIL}</b>     | In-built parameter that contains the object of application utility                                                                                                                                                                                                                                                                                         |



|  |                                                                                                        |
|--|--------------------------------------------------------------------------------------------------------|
|  | class (FinAppUtil).This should be passed as it is. This object is responsible for invoking the script. |
|--|--------------------------------------------------------------------------------------------------------|

\*The “input data” may contain either name value pairs of the input repository fields, or just names of report variables. Wherever only report variable name is encountered, the method will retrieve value of the variable from the current state of report execution, and populate this value in the corresponding input repository field of the script.

### Logic:

This method performs the following tasks:

- Populate “Input Repository Fields” of the script using the “input data” provided. The fields are populated in the repository class BANCS.INPUT

- Execute the script in current Finacle context

- Populate the report variables present in “output data” with values of corresponding “Output Repository Fields” of the script (from the repository class BANCS.OUTPUT)

### Sample Usage:

```
FincoreUtil.executeScript( "scriptname.scr",  
"inpField1=val1|inpField2=val2|inpField3|inpField4"  
"reportOutVar1|reportOutVar2",  
${REPORT_SCRIPTLET},  
${FIN_APP_UTIL} );
```

Above example contains a call to the script “scriptname.scr”. The input repository variables of this script will be populated as follows:

| Repository variable name | Value populated |
|--------------------------|-----------------|
|                          |                 |

|                              |                                      |
|------------------------------|--------------------------------------|
| <b>BANCS.INPUT.inpField1</b> | val1                                 |
| <b>BANCS.INPUT.inpField2</b> | val2                                 |
| <b>BANCS.INPUT.inpField3</b> | <Value of report variable inpField3> |
| <b>BANCS.INPUT.inpField4</b> | <Value of report variable inpField4> |

And the output report variables will be populated after script execution as follows:

| Repository variable name | Value populated                                         |
|--------------------------|---------------------------------------------------------|
| <b>reportOutVar1</b>     | Value of repository variable BANCS.OUTPUT.reportOutVar1 |
| <b>reportOutVar2</b>     | Value of repository variable BANCS.OUTPUT.reportOutVar2 |

© EdgeVerve Systems Limited

Confidential

External

## Appendix

The fields values available for scripts in the repository BANCS and class STDIN are as follows:

| Sl. No. | Field Name             | Value Contained                                                                                                                                                                   |
|---------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.      | " languageCode"        | <p>This field contains the value of the language code of the user.</p> <hr/> <p><b>For Example:</b></p> <p>INFENG</p> <hr/>                                                       |
| 2.      | " userId"              | The user ID of the user who executed the script.                                                                                                                                  |
| 3.      | " onlineOrBatch"       | Whether this script is being executed through a batch program or online ( "O" or "B").                                                                                            |
| 4.      | " userWorkClass"       | <p>The workclass of the user ID who executed the script from HUPM.</p> <p>See <a href="#">Maintain User Profile</a> in the Single Signon module for HUPM menu option details.</p> |
| 5.      | " menuOption"          | The menu option which called this script.                                                                                                                                         |
| 6.      | " homeCrncyCode"       | The home currency of the data center from HSCFM.                                                                                                                                  |
| 7.      | " homeCrncyAlias"      | The home currency alias.                                                                                                                                                          |
| 8.      | " CurrentBancsVersion" | The Finacle version.                                                                                                                                                              |
| 9.      | " myBankCode"          | The bank code of the database.                                                                                                                                                    |

|     |                   |                                                                                                                                                      |
|-----|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10. | " myBrCode"       | The branch code of the SOL.                                                                                                                          |
| 11. | " myExtCode"      | The Extension counter.                                                                                                                               |
| 12. | " mySolId"        | The SOLID .                                                                                                                                          |
| 13. | " mySolAlias"     | The SOLALIAS.                                                                                                                                        |
| 14. | " mySolDesc"      | The description for the Sol as specified in HSCFM.                                                                                                   |
| 15. | " homeSolId"      | The Sol to which the user belongs.                                                                                                                   |
| 16. | " homeSolAlias"   | The Home SOL Alias.                                                                                                                                  |
| 17. | " homeSolDesc"    | The Home Sol description as specified in HSCFM.                                                                                                      |
| 18. | " dcAlias"        | The DC ALIAS.                                                                                                                                        |
| 19. | " SBString"       | The value of custoption for SBSTING.                                                                                                                 |
| 20. | " CASTring"       | The value of custoption for CASTING.                                                                                                                 |
| 21. | " LLString"       | The value of custoption for LLSTING.                                                                                                                 |
| 22. | " CCString"       | The value of custoption for CCSTING.                                                                                                                 |
| 23. | " sysDate"        | The system date of the machine.                                                                                                                      |
| 24. | " BODDate"        | The current BOD date of the SOL.                                                                                                                     |
| 25. | " termClass"      | The terminal class from HTPM.<br><i>See Add Terminal and Configure Device For Terminal in the Administrator module for HTPM menu option details.</i> |
| 26. | " moduleIdentity" | The module which is calling the script.                                                                                                              |
| 27. | " TestFlg"        | Whether the script is being invoked in test mode.<br><br><b>For Example:</b><br><br>Through menu option script                                       |
| 28. | " WFflg"          | Whether the script is a workflow script.                                                                                                             |
| 29. | "ScriptName"      | The name of the script.                                                                                                                              |

**Note:**

*See Maintain SOL ID in the Administrator module for HSCFM menu option details.*

*See Maintain Bank in the Administrator module for HSCFM menu option details.*

© EdgeVerve Systems Limited

External

Confidential

## Summary

---

Scripting is a programming language used to customize the application to suit the banks' requirement. This is a part of the extensibility tool kit provided in Finacle.

Like any other programming tool, scripting has its own syntax. All scripts must end with an extension .scr and must exist in the particular directories only below the Finacle root directory.

Every script must have a start tag. Scripting has built-in functions. Scripting provides certain built-in functions called as user hooks. Scripting has its own control statements and operators to validate statements.

Scripts store the values in repositories. These repositories are divided into classes and these classes contain the variable name and value pair.

Scripting language provides certain debugging utilities like trace on and print to debug the script.

## Manipulate Repository Variables

---

The state of information is stored in repository variables only. To give more flexibility for the application programmer, scripting allows the programmers to access these repository fields just like any other built-in variables.

The programmer can create new fields in the repository dynamically by giving new field names and values to them, though the type of field is fixed depending on the class type.

All the repository fields must be accessed by specifying

REPOSITORY-NAME.CLASS-NAME.FIELD-NAME

Repository variables REPOSITORY-NAME.CLASS-NAME.FIELD-NAME can be given as

(Expression1).(Expression2).(Expression3), where Expression1, Expression2 and Expression3 reduce to strings

---

### For Example:

```
if sv_a = "hello"
```

```
then (" t" + " e" + "s" + " t").(" str" + " ing").(sv_a)
```

will reduce to test.string.hello

REP.CLASS.FIELD, where Rep, Class and Field are the Repository Name, Class Name and Field Name respectively.

All the repository fields are globally accessible across the scripts.

```
<--start
#ACCESS repository fields in one of the following ways
#assign it to a scratchpad variable
sv_a = MYREP.integerCLASS.FIELD1
#sv_a will become an INTEGER type variable
#access it directly in statements
IF (MYREP.integerCLASS.FIELD1 == 1234)
# do something
ENDIF

#MODIFY repository fields by directly assigning a
literal/expression
sv_a = "Hello World"
MYREP.stringCLASS.FIELD2 = sv_a
MYREP.stringCLASS.FIELD2 = "Hello World"
MYREP.stringCLASS.FIELD2 = "Hello" + "World"
#all the above statements modify or create FIELD2
#Create a new field in a repository in this way
MYREP.stringCLASS.NEWFIELD1 = "Hello World"
MYREP.integerCLASS.NEWFIELD2 = 123
end-->
```

---

Built in repository functions are provided for checking if the Repository / Class / Field exists.

See [Built-in Utility Functions](#) for details on in-built utilities for *REPEXISTS*, *CLASSEXISTS*, *FIELDEXISTS*, *CREATEREP*, *CREATECLASS*, *DELETEREP* and *DELETECLASS*.



## Example

---

```
<--start
# Checking MID$ operation
sv_a = Mid$("1234567890",3,5)
print (sv_a)
# Prints "45678"
# Checking LEFT$ operation
sv_a = Left$("1234567890",3)
print (sv_a)
# Prints "123"
# Checking RIGHT$ operation
sv_a = Right$("1234567890",4)
print (sv_a)
# Prints "7890"
# Checking CINT operation
sv_a = CINT("123456")
print (sv_a)
# Prints "123456"
# Checking CDOUBLE operation
sv_a = CDOUBLE("1234.5678")
print (sv_a)
# Prints "1234.567800"
# Checking TOLOWER operation
```

```
sv_a = TOLOWER(" kfsjlLKJLZVXLkljsl")
print (sv_a)
# Prints " kfsjllkjlzvxkljsl"
# Checking TOUPPER operation
sv_a = TOUPPER("werQWIUsd5678")
print (sv_a)
# Prints "WERQWIUSD5678"
# Checking the FORMAT$ operations
sv_s = Format$(12, "% d")
print (sv_s)
# Prints "12"
sv_s = Format$(1454, "% ld")
print (sv_s)
# Prints "1454"
sv_s = Format$(12.3, "%2.3f")
print (sv_s)
# Prints "12.300"
# Checking the SET$ operations
sv_s = Set$("1234567890", 3, 5, " ASDFG")
print (sv_s)
# Prints "123ASDFG90"
sv_s = Set$("1234567890", 3, 3, " ASDFG")
print (sv_s)
# Prints "123ASD7890"
sv_s = Set$("1234567890", 7, 5, " ASDFG")
print (sv_s)
# Prints "1234567ASD"
# Checking ENCRYPT operation
```

```
sv_n = "%%vi nay"
sv_u = encrypt ( sv_n)
print ( sv_u)
# Prints "%25%25vi+nay"
# Checking STRLEN operation
sv_n = "Some Useless Text"
sv_u = STRLEN ( sv_n)
print ( sv_u)
# Prints "17"
# Checking CHARAT operation
sv_n = "Some Text"
sv_u = CHARAT ( sv_n, 5)
print ( sv_u)
# Prints " T"
# Checking RTRIM operation
sv_a = "AS "
sv_b = rtrim(sv_a)
print (sv_b)
# Prints "AS"
sv_a = " AS*****"
sv_b = rtrim(sv_a, '*')
print (sv_b)
# Prints "AS"
# Checking LTRIM operation
sv_a = " AS"
sv_b = ltrim(sv_a)
print (sv_b)
# Prints "AS"
```

```

sv_a = "****AS"
sv_b = ltrim(sv_a, '*')
print (sv_b)
# Prints "AS"
# Checking TRIM operation
sv_a = " AS "
sv_b = trim(sv_a)
print (sv_b)
# Prints "AS"
sv_a = "**** AS****"
sv_b = trim(sv_a, '*')
print (sv_b)
# Prints "AS"
# Checking LPAD operation
sv_a = "AS"
sv_b = lpad(sv_a, 10, '*')
sv_c = sv_b + "END"
print (sv_c)
# Prints "***** ASEND"
# Checking RPAD operation
sv_a = "AS"
sv_b = rpad(sv_a, 10, '*')
sv_c = sv_b + "END"
print (sv_c)
# Prints " AS*****END"
# Checking the basic Repository / Class create / delete operations
# Repository " myrep" is created
if ( REPEXISTS(" myrep") == 0) then

```

```

    CREATEREP (" myrep")
endif
# Class " myclass" is created
if ( CLASSEXISTS(" myrep", " myclass") == 0) then
    CREATECLASS (" myrep", " myclass", 1)
endif
# Class " myclass" is created
if ( FIELDEXISTS(myrep.myclass.myfield) == 0) then
    print (" myrep.myclass.myfield doesn't exist")
    myrep.myclass.myfield = 102
    print ( myrep.myclass.myfield)
endif
# Class " myclass" is deleted
DELETECLASS (" myrep", " myclass")
# Repository " myrep" is deleted
DELETEREP (" myrep")
# Checking GETPOSITION operation
sv_a = "Some very long absurd text"
sv_b = "long"
sv_c = getposition (sv_a, sv_b)
print (sv_c)
#prints "11"
sv_b = "LONG"
sv_c = getiposition (sv_a, sv_b)
print (sv_c)
#prints "11"
# Checking STRICMP operation
sv_a = "Hello"

```

```
sv_b = "hello"
if(STRICMP(sv_a, sv_b) == 1) then
    print ("The strings are equal")
else
    print ("The strings are not equal")
endif
```

```
# Checking GETSTRING operation
sv_a = 'A'
sv_b = GETSTRING (sv_a)
print (sv_b)
#prints "A"
# checking PUTREPFIELD operation
sv_n = "Checking PUTREPFIELD operation"
print (sv_n)
CREATEREP("TstRep")
CREATECLASS("TstRep","TstClass",5)
TstRep.TstClass.Temp = "TstRep.TstClass.TstFld"
sv_a = "TstRep.TstClass.TstFld"
sv_b = "Value"
PUTREPFIELD (sv_a ,sv_b)
# checking GETREPFIELD operation
sv_n = "Checking GETREPFIELD operation"
print (sv_n)
TstRep.TstClass.Temp = "TstRep.TstClass.TstFld"
sv_a = "TstRep.TstClass.TstFld"
sv_b = GETREPFIELD(sv_a)
print (sv_b)
```

---

## About Infosys Finacle

Finacle is the industry-leading digital banking solution suite from EdgeVerve Systems, a wholly owned product subsidiary of Infosys. Finacle helps traditional and emerging financial institutions drive truly digital transformation to achieve frictionless customer experiences, larger ecosystem play, insights-driven interactions and ubiquitous automation. Today, banks in over 100 countries rely on Finacle to service more than a billion consumers and 1.3 billion accounts.

Finacle solutions address the core banking, omnichannel banking, payments, treasury, origination, liquidity management, Islamic banking, wealth management, analytics, artificial intelligence, and blockchain requirements of financial institutions to drive business excellence. An assessment of the top 1250 banks in the world reveals that institutions powered by the Finacle Core Banking solution, on average, enjoy 7.2% points lower costs-to-income ratio than others.



---

For more information, contact [finacle@edgeverve.com](mailto:finacle@edgeverve.com)

[www.finacle.com](http://www.finacle.com)

©2020 EdgeVerve Systems Limited, a wholly owned subsidiary of Infosys, Bangalore, India. All Rights Reserved. This documentation is the sole property of EdgeVerve Systems Limited ("EdgeVerve"). EdgeVerve believes the information in this document or page is accurate as of its publication date; such information is subject to change without notice. EdgeVerve acknowledges the proprietary rights of other companies to the trademarks, product names and such other intellectual property rights mentioned in this document. This document is not for general distribution and is meant for use solely by the person or entity that it has been specifically issued to and can be used for the sole purpose it is intended to be used for as communicated by EdgeVerve in writing. Except as expressly permitted by EdgeVerve in writing, neither this documentation nor any part of it may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, printing, photocopying, recording or otherwise, without the prior written permission of EdgeVerve and/or any named intellectual property rights holders under this document.